ABISHEK DEVARAJ

192512126

Experiment No. 5

IoE-Based Smart Greenhouse Monitoring System — Google Colab Simulation

AIM

To simulate sensor onboarding, authentication, cloud data transmission, and stream analytics for an IoE-based Smart Greenhouse Monitoring System using Python in Google Colab.

OBJECTIVES

- Simulate IoE greenhouse sensor registration.
- Authenticate greenhouse sensor devices.
- Generate real-time environmental sensor data.
- Transmit sensor telemetry to a simulated cloud.
- Perform stream analytics on sensor data.
- Detect dry soil conditions.
- Detect high-temperature conditions.
- Detect low-humidity conditions.
- Visualize sensor data using graphs.
- Store collected cloud data in CSV format.

SOFTWARE REQUIRED

- Google Colab
- Python 3
- pandas
- numpy
- matplotlib

THEORY

The Internet of Everything (IoE) connects **people, processes, data, and things** through communication networks and intelligent systems.

In a Smart Greenhouse, IoE enables connected sensors to continuously collect environmental information such as:

- Temperature
- Humidity
- Soil Moisture
- Zone ID
- Timestamp

The typical IoE data flow is:

```
Greenhouse Sensors
        ↓
Device Registration
        ↓
Device Authentication
        ↓
IoE / Cloud Platform
        ↓
Cloud Data Storage
        ↓
Stream Analytics
        ↓
Alert / Decision
```

In this experiment, Google Colab simulates:

- Sensor-device onboarding
- Device authentication
- Sensor-data generation
- Cloud upload
- Stream analytics

- Dry-soil detection
- Temperature anomaly detection
- Humidity anomaly detection

ALGORITHM

1. Start the program.
2. Create smart greenhouse sensor devices.
3. Register the devices.
4. Authenticate each device.
5. Generate temperature, humidity, and soil-moisture data.
6. Send the telemetry data to the simulated cloud.
7. Store the received data in a DataFrame.
8. Calculate average temperature, humidity, and soil moisture.
9. Detect dry soil conditions.
10. Detect high-temperature conditions.
11. Detect low-humidity conditions.
12. Generate IoE alerts.
13. Perform device-wise analysis.
14. Plot sensor data.
15. Save the cloud data as a CSV file.
16. Stop the program.

GOOGLE COLAB CODE:

BEST GRAPH - AVERAGE SOIL MOISTURE BY ZONE

Calculate average soil moisture for each zone

zone_soil = df.groupby("Device_ID")["Soil_Moisture"].mean().reset_index()

Shorten device names for graph

zone_soil["Zone"] = zone_soil["Device_ID"].str.replace("GREENHOUSE_ZONE_", "ZONE ")

plt.figure(figsize=(10, 6))

bars = plt.bar(zone_soil["Zone"], zone_soil["Soil_Moisture"])

Irrigation threshold

plt.axhline(y=40, linestyle="--", linewidth=2, label="Irrigation Threshold (40%)")

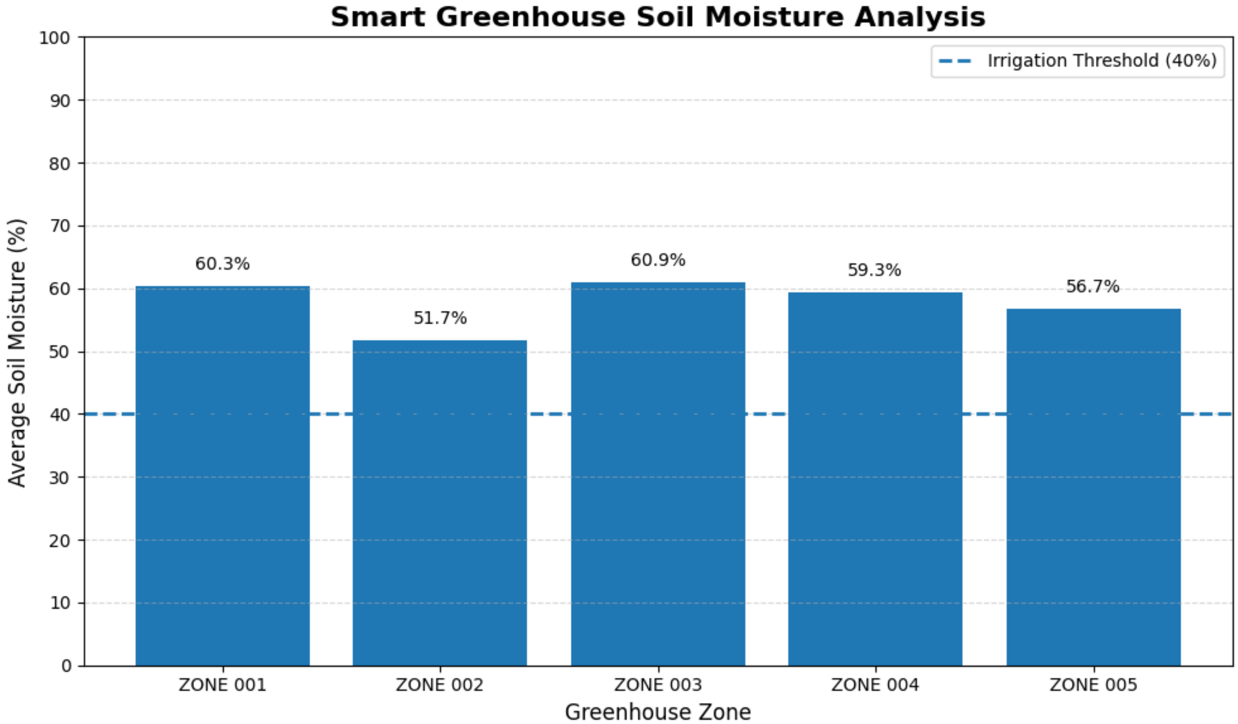
Display percentage above each bar

for bar in bars:

```
height = bar.get_height()

plt.text(
    bar.get_x() + bar.get_width() / 2,
    height + 2,
    f"{height:.1f}%",
    ha="center",
    va="bottom",
    fontsize=10
```

```
)

plt.title( "Smart Greenhouse Soil Moisture Analysis", fontsize=16, fontweight="bold" )

plt.xlabel( "Greenhouse Zone", fontsize=12 )

plt.ylabel( "Average Soil Moisture (%)", fontsize=12 )

plt.ylim(0, 100)

plt.yticks( range(0, 101, 10) )

plt.grid( axis="y", linestyle="--", alpha=0.5 )

plt.legend()

plt.tight_layout()

plt.show()
```

OUTPUT:



RESULT

The **IoE-based Smart Greenhouse Monitoring System** was successfully simulated using Google Colab. Greenhouse sensor devices were registered and authenticated, temperature, humidity, and soil-moisture telemetry was generated and transmitted to the simulated cloud environment, and stream analytics were performed. Dry-soil, high-temperature, and low-humidity conditions were detected, appropriate IoE alerts were generated, sensor data was visualized using graphs, and the collected cloud data was successfully stored in CSV format.